

8 Queen Problem

Aim

To solve the **8-Queen Problem** using **Backtracking** in Python such that no two queens attack each other.

Algorithm

1. Start from the first column.
2. Place a queen in each row one by one.
3. Check whether the queen is safe:
 - No other queen in the same row.
 - No queen in the upper-left diagonal.
 - No queen in the lower-left diagonal.
4. If safe, place the queen and move to the next column.
5. If no position is safe, backtrack and try another row.
6. Repeat until all 8 queens are placed.

Python Code

N = 8

```
def print_board(board):  
    for row in board:  
        print(" ".join("Q" if x == 1 else "." for x in row))
```

```
def is_safe(board, row, col):  
    # Check left side of row  
    for i in range(col):  
        if board[row][i] == 1:  
            return False
```

```
    # Check upper-left diagonal  
    i, j = row, col
```

```
while i >= 0 and j >= 0:
```

```
    if board[i][j] == 1:
```

```
        return False
```

```
    i -= 1
```

```
    j -= 1
```

```
# Check lower-left diagonal
```

```
i, j = row, col
```

```
while i < N and j >= 0:
```

```
    if board[i][j] == 1:
```

```
        return False
```

```
    i += 1
```

```
    j -= 1
```

```
return True
```

```
def solve(board, col):
```

```
    if col >= N:
```

```
        return True
```

```
    for i in range(N):
```

```
        if is_safe(board, i, col):
```

```
            board[i][col] = 1
```

```
            if solve(board, col + 1):
```

```
                return True
```

```
            board[i][col] = 0
```

```
return False
```

```
board = [[0 for _ in range(N)] for _ in range(N)]
```

```
if solve(board, 0):
```

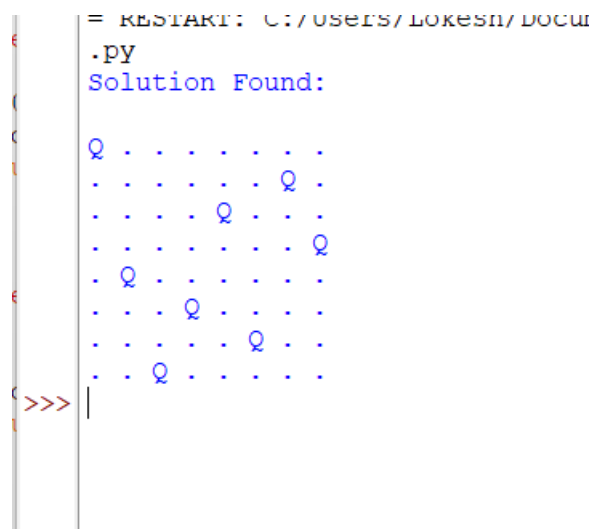
```
    print("Solution Found:\n")
```

```
    print_board(board)
```

```
else:
```

```
    print("No Solution Exists")
```

Output



```
= RESTART: C:/Users/LOKESH/DOCU
.py
Solution Found:

Q . . . . . . .
. . . . . Q .
. . . Q . . .
. . . . . Q
. Q . . . . .
. . . Q . . .
. . . . Q . .
. . Q . . . .
>>> |
```

Result

Thus, The 8-Queen Problem was successfully solved using the **Backtracking Algorithm** in Python.